

Árbol Binario en C++

Generated by Doxygen 1.16.1

1 Árbol Binario en C++	1
1.1 Características	1
1.2 Estructura del proyecto	1
1.3 Documentación	2
1.4 Ejecución	2
1.5 Autora	2
1.6 Maestro	2
1.7 Materia y Programa Académico	2
1.8 Fecha	2
2 Pruebas	3
2.1 Resultados de pruebas	3
3 Directory Hierarchy	7
3.1 Directories	7
4 Hierarchical Index	9
4.1 Class Hierarchy	9
5 Class Index	11
5.1 Class List	11
6 File Index	13
6.1 File List	13
7 Directory Documentation	15
7.1 headers Directory Reference	15
7.2 src Directory Reference	15
8 Class Documentation	17
8.1 Arbol< T > Class Template Reference	17
8.1.1 Detailed Description	17
8.1.2 Constructor & Destructor Documentation	18
8.1.2.1 Arbol()	18
8.1.2.2 ~Arbol()	18
8.1.3 Member Function Documentation	18
8.1.3.1 buscarMax()	18
8.1.3.2 contarHojas()	18
8.1.3.3 contarInternos()	19
8.1.3.4 contarNodos()	19
8.1.3.5 insertarDato()	19
8.1.3.6 orden()	20
8.1.3.7 promedio()	20
8.1.3.8 visualizacion()	21
8.2 excepcion Class Reference	22

8.2.1 Detailed Description	22
8.2.2 Constructor & Destructor Documentation	22
8.2.2.1 excepcion()	22
8.3 Nodo< T > Class Template Reference	22
8.3.1 Detailed Description	23
8.3.2 Constructor & Destructor Documentation	23
8.3.2.1 Nodo() [1/3]	23
8.3.2.2 Nodo() [2/3]	23
8.3.2.3 Nodo() [3/3]	24
8.3.3 Member Data Documentation	24
8.3.3.1 nodo_der	24
8.3.3.2 nodo_izq	24
8.3.3.3 valor	24
9 File Documentation	25
9.1 headers/arbol.h File Reference	25
9.1.1 Detailed Description	25
9.2 arbol.h	26
9.3 headers/excepcion.h File Reference	26
9.3.1 Detailed Description	26
9.4 excepcion.h	27
9.5 headers/nodo.h File Reference	27
9.5.1 Detailed Description	27
9.6 nodo.h	27
9.7 README.md File Reference	28
9.8 src/arbol.cpp File Reference	28
9.8.1 Detailed Description	28
9.9 arbol.cpp	28
9.10 src/main.cpp File Reference	32
9.10.1 Detailed Description	32
9.10.2 Function Documentation	33
9.10.2.1 main()	33
9.10.2.2 menu()	34
9.11 main.cpp	34
9.12 src/nodo.cpp File Reference	36
9.12.1 Detailed Description	36
9.13 nodo.cpp	36
Index	37

Chapter 1

Árbol Binario en C++

Implementación de un árbol binario en C++ con funciones de inserción, búsqueda, visualización y análisis estadístico de la información almacenada en la estructura.

1.1 Características

- Inserción de nodos.
- Conteo de estructura.
- Análisis de datos.
- Visualización selectiva.
- Destructor recursivo.

1.2 Estructura del proyecto

- doc/
 - html/
 - latex/
- images/
 - Imágenes de resultados de pruebas.
- src/
 - [arbol.cpp](#)
 - [nodo.cpp](#)
 - [main.cpp](#)
- headers/
 - [arbol.h](#)
 - [nodo.h](#)
 - [excepcion.h](#)

1.3 Documentación

Toda la documentación ha sido generada con Doxygen.

Para visualizar: Diríjase a `doc/html/index.html` Posteriormente, abra con su navegador de preferencia.

Para visualizar todos los comentarios: Vaya a "Class Members", después, seleccione "All".

O abra el archivo PDF "refman.pdf".

1.4 Ejecución

- Usando comandos: Abra su terminal, y ejecute el siguiente código...

```
g++ src/main.cpp -Iheaders -o binArbol ./binArbol
```

1.5 Autora

Camila Andrea Jaimes Martinez.

1.6 Maestro

Manuel Balderas Victoria.

1.7 Materia y Programa Académico

- Estructura de Datos (Aula).
- Ingeniería en Software y Minería de Datos (1er año).

1.8 Fecha

Miercoles 22 de Abril, 2026.

Chapter 2

Pruebas

¡Ejecución del programa!

2.1 Resultados de pruebas

Las siguientes imágenes muestran la ejecución exitosa del programa correspondiente al árbol binario, el cual tiene las funciones...

- Inserción de nodos.
- Conteo de la estructura.
- Análisis de datos.
- Visualización selectiva.

```
camila@MacBook-Air-de-Camila bin_arbol % ./binArbol
[

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 1

¿Qué valor desea agregar al árbol?: 56
¡Se ha añadido con éxito el nodo con valor 56!

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 1

¿Qué valor desea agregar al árbol?: 12
¡Se ha añadido con éxito el nodo con valor 12!

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 1

¿Qué valor desea agregar al árbol?: 100
¡Se ha añadido con éxito el nodo con valor 100!

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 1
```

Figure 2.1 Ejecución del Árbol Binario (Parte 1)

```
--> Seleccione una opción: 1

¿Qué valor desea agregar al árbol?: 91
¡Se ha añadido con éxito el nodo con valor 91!

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 1

¿Qué valor desea agregar al árbol?: 78
¡Se ha añadido con éxito el nodo con valor 78!

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 1

¿Qué valor desea agregar al árbol?: 34
¡Se ha añadido con éxito el nodo con valor 34!

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 1

¿Qué valor desea agregar al árbol?: 25
¡Se ha añadido con éxito el nodo con valor 25!

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.
```

Figure 2.2 Ejecución del Árbol Binario (Parte 2)

```
6) Salir del programa.

--> Seleccione una opción: 2

Número total de NODOS: 7
Número total de NODOS INTERNOS: 4

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 3

Valor MÁXIMO en el árbol: 100
PROMEDIO de los nodos : 56.5714

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 4

Nivel 0 = (Raíz) 56 |
Nivel 1 = (Interno) 12 | (Interno) 100 |
Nivel 2 = (Interno) 34 | (Interno) 91 |
Nivel 3 = (Hoja) 25 | (Hoja) 78 |

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 5

--- PRE-ORDEN ---
56 | 12 | 34 | 25 | 100 | 91 | 78 |

--- IN-ORDEN ---
12 | 34 | 25 | 56 | 100 | 91 | 78 |

--- POST-ORDEN ---
12 | 34 | 25 | 100 | 91 | 78 | 56 |

--- MENU! ---
```

Figure 2.3 Ejecución del Árbol Binario (Parte 3)


```

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 3

Valor MÁXIMO en el árbol: 100
PROMEDIO de los nodos : 56.5714

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 4
Nivel 0 = (Raiz) 56 |
Nivel 1 = (Interno) 12 | (Interno) 100 |
Nivel 2 = (Interno) 34 | (Interno) 91 |
Nivel 3 = (Hoja) 25 | (Hoja) 78 |

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 5

--- PRE-ORDEN ---
56 | 12 | 34 | 25 | 100 | 91 | 78 |
--- IN-ORDEN ---
12 | 34 | 25 | 56 | 100 | 91 | 78 |
--- POST-ORDEN ---
12 | 34 | 25 | 100 | 91 | 78 | 56 |

--- MENU! ---

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 6

camila@MacBook-Air-de-Camila bin_arbol %

```

Figure 2.4 Ejecución del Árbol Binario (Parte 4)

Si se usan excepciones, dado el caso que el árbol se encuentre vacío, el programa lanzará un mensaje de error, como se muestra a continuación...

```

-

---- MENU! ----

1) Insertar un nodo.
2) Conteo de estructura (Total de nodos y de nodos internos).
3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).
4) Visualización selectiva (Información de los nodos hojas y de internos).
5) Visualización en pre-orden, in-orden y post-orden.
6) Salir del programa.

--> Seleccione una opción: 2

¡El árbol se encuentra vacío!

```

Figure 2.5 Uso de excepción (Parte 5 - Final)

Chapter 3

Directory Hierarchy

3.1 Directories

headers	15
arbol.h	25
excepcion.h	26
nodo.h	27
src	15
arbol.cpp	28
main.cpp	32
nodo.cpp	36

Chapter 4

Hierarchical Index

4.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Arbol< T >	17
Nodo< T >	22
std::runtime_error	
excepcion	22

Chapter 5

Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Arbol< T >	17
excepcion	22
Nodo< T >	22

Chapter 6

File Index

6.1 File List

Here is a list of all files with brief descriptions:

headers/ arbol.h	Define la estructura del árbol binario	25
headers/ excepcion.h	Define la clase "excepción". La clase excepción permite la gestión de excepciones que pueda presentar la lista	26
headers/ nodo.h	Define la estructura de un Nodo	27
src/ arbol.cpp	Implementación de los constructores y métodos de la plantilla Arbol	28
src/ main.cpp	El programa completo del árbol binario	32
src/ nodo.cpp	Implementación de los constructores y métodos de la plantilla Nodo	36

Chapter 7

Directory Documentation

7.1 headers Directory Reference

Files

- file [arbol.h](#)
Define la estructura del árbol binario.
- file [excepcion.h](#)
Define la clase "excepción". La clase excepción permite la gestión de excepciones que pueda presentar la lista.
- file [nodo.h](#)
Define la estructura de un [Nodo](#).

7.2 src Directory Reference

Files

- file [arbol.cpp](#)
Implementación de los constructores y métodos de la plantilla [Arbol](#).
- file [main.cpp](#)
El programa completo del árbol binario.
- file [nodo.cpp](#)
Implementación de los constructores y métodos de la plantilla [Nodo](#).

Chapter 8

Class Documentation

8.1 Arbol< T > Class Template Reference

```
#include <arbol.h>
```

Public Member Functions

- [Arbol](#) ()
Inicializa el árbol vacío.
- [~Arbol](#) ()
Destruye el árbol, liberando la memoria asignada a este.
- void [insertarDato](#) (T dato)
Inserta un nuevo nodo con el valor de "dato" en el árbol.
- int [contarNodos](#) ()
Función pública que llama a contarNodos(Nodo< T> inicio).*
- int [contarHojas](#) ()
Función pública que llama a contarHojas(Nodo< T> inicio).*
- int [contarInternos](#) ()
Función pública que llama a contarInternos(Nodo< T> inicio).*
- T [buscarMax](#) ()
Función pública que llama a buscarMax(Nodo< T> inicio).*
- double [promedio](#) ()
Calcula el promedio de los valores almacenados en los nodos del árbol.
- void [visualizacion](#) ()
Imprime el contenido de los nodos por nivel.
- void [orden](#) ()
Imprime el contenido de los nodos en pre-orden, in-orden y post-orden.

8.1.1 Detailed Description

```
template<typename T>  
class Arbol< T >
```

Definition at line 15 of file [arbol.h](#).

8.1.2 Constructor & Destructor Documentation

8.1.2.1 Arbol()

```
template<typename T>
Arbol< T >::Arbol ()
```

Inicializa el árbol vacío.

La referencia a la raíz del árbol se establece a nullptr.

Definition at line 22 of file [arbol.cpp](#).

```
00023 {
00024     raiz = nullptr;
00025 }
```

8.1.2.2 ~Arbol()

```
template<typename T>
Arbol< T >::~~Arbol ()
```

Destruye el árbol, liberando la memoria asignada a este.

Destructor del árbol que llama a la función `eliminarArbol(Nodo<T>* inicio)` para liberar la memoria asignada al árbol binario.

Definition at line 53 of file [arbol.cpp](#).

```
00054 {
00055     eliminarArbol(raiz);
00056 }
```

8.1.3 Member Function Documentation

8.1.3.1 buscarMax()

```
template<typename T>
T Arbol< T >::buscarMax ()
```

Función pública que llama a `buscarMax(Nodo<T>* inicio)`.

Definition at line 246 of file [arbol.cpp](#).

```
00247 {
00248     return buscarMax(raiz);
00249 }
```

8.1.3.2 contarHojas()

```
template<typename T>
int Arbol< T >::contarHojas ()
```

Función pública que llama a `contarHojas(Nodo<T>* inicio)`.

Definition at line 163 of file [arbol.cpp](#).

```
00164 {
00165     return contarHojas(raiz);
00166 }
```

8.1.3.3 contarInternos()

```
template<typename T>
int Arbol< T >::contarInternos ()
```

Función pública que llama a contarInternos(Nodo<T>* inicio).

Definition at line 205 of file [arbol.cpp](#).

```
00206 {
00207     return contarInternos(raiz, raiz);
00208 }
```

8.1.3.4 contarNodos()

```
template<typename T>
int Arbol< T >::contarNodos ()
```

Función pública que llama a contarNodos(Nodo<T>* inicio).

Definition at line 125 of file [arbol.cpp](#).

```
00126 {
00127     return contarNodos(raiz);
00128 }
```

8.1.3.5 insertarDato()

```
template<typename T>
void Arbol< T >::insertarDato (
    T dato)
```

Inserta un nuevo nodo con el valor de "dato" en el árbol.

El nuevo nodo se inserta en la "raiz" dado el caso que esta se encuentre vacia. De lo contrario, se insertará en el subárbol izquierdo si el valor del nuevo nodo es inferior al de la raíz, o en el subárbol derecho si es superior.

Parameters

<i>dato</i>	El valor que se almacenará en el nuevo nodo del árbol.
-------------	--

Definition at line 70 of file [arbol.cpp](#).

```
00071 {
00072     Nodo<T>* nuevo_nodo = new Nodo<T>(dato);
00073     if (raiz == nullptr)
00074     {
00075         raiz = nuevo_nodo;
00076     }
00077     else
00078     {
00079         Nodo<T>* anterior = nullptr;
00080         Nodo<T>* inicio = raiz;
00081
00082         while(inicio != nullptr)
00083         {
00084             anterior = inicio;
00085             inicio = (dato < inicio->valor) ? inicio->nodo_izq : inicio->nodo_der;
00086         }
00087
00088         if (dato < anterior->valor)
00089         {
00090             anterior->nodo_izq = nuevo_nodo;
00091         }
00092         else
00093         {
00094             anterior->nodo_der = nuevo_nodo;
00095         }
00096     }
00097
00098     cout << "¡Se ha añadido con éxito el nodo con valor " << dato << "!\n";
00099 }
```

8.1.3.6 orden()

```
template<typename T>
void Arbol< T >::orden ()
```

Imprime el contenido de los nodos en pre-orden, in-orden y post-orden.

Es la función de acceso público que utiliza la función privadas que permiten la obtención del contenido del árbol en diferentes ordenes, es decir, ordenPre(Nodo<T>* inicio), ordenIn(Nodo<T>* inicio) y ordenPost(Nodo<T>* inicio).

Dado el caso que el árbol se encuentre vacío, se lanza una excepción.

Definition at line 440 of file [arbol.cpp](#).

```
00441 {
00442     if (raiz == nullptr)
00443     {
00444         throw excepcion("El árbol se encuentra vacío!");
00445     }
00446
00447     cout << "\n--- PRE-ORDEN ---\n";
00448     ordenPre(raiz);
00449
00450     cout << "\n\n--- IN-ORDEN ---\n";
00451     ordenIn(raiz);
00452
00453     cout << "\n\n--- POST-ORDEN ---\n";
00454     ordenPost(raiz);
00455 }
```

8.1.3.7 promedio()

```
template<typename T>
double Arbol< T >::promedio ()
```

Calcula el promedio de los valores almacenados en los nodos del árbol.

Devuelve la suma de los valores almacenados en los nodos dividida por el número total de nodos en el árbol.

Dado el caso que el árbol se encuentre vacío, se lanza una excepción.

Definition at line 281 of file [arbol.cpp](#).

```
00282 {
00283     if (raiz == nullptr)
00284     {
00285         throw excepcion("El árbol se encuentra vacío!");
00286     }
00287
00288     double suma = sumaNodos(raiz);
00289     double num = contarNodos();
00290
00291     return suma / num;
00292 }
```


8.1.3.8 visualizacion()

```
template<typename T>
void Arbol< T >::visualizacion ()
```

Imprime el contenido de los nodos por nivel.

Se organizan los nodos almacenados en el árbol dentro de una cola para mostrarlos por nivel; gracias al bucle while, primero imprime el número del nivel actual, seguido de los nodos y la categoría a la que pertenecen (raíz, hoja u interno). El uso de !cola.empty() asegura el no perder ningún nodo del árbol.

Dado el caso que el árbol se encuentre vacío, se lanza una excepción.

Definition at line 307 of file [arbol.cpp](#).

```
00308 {
00309     if (raiz == nullptr)
00310     {
00311         throw excepcion("El árbol se encuentra vacío!");
00312     }
00313
00314     queue<Nodo<T>*> cola;
00315     cola.push(raiz);
00316
00317     int nivel = 0;
00318
00319     while (!cola.empty())
00320     {
00321         int elementosNivel = cola.size();
00322         cout << "Nivel " << nivel << " = ";
00323
00324         for (int i = 0; i < elementosNivel; i++)
00325         {
00326
00327             Nodo<T>* nodo_actual = cola.front();
00328             cola.pop();
00329
00330             string categoria;
00331
00332             if (nodo_actual == raiz)
00333             {
00334                 categoria = "(Raíz)";
00335             }
00336             else if (nodo_actual->nodo_izq == nullptr && nodo_actual->nodo_der == nullptr)
00337             {
00338                 categoria = "(Hoja)";
00339             }
00340             else
00341             {
00342                 categoria = "(Interno)";
00343             }
00344
00345             cout << categoria << " " << nodo_actual->valor << " | ";
00346
00347             if (nodo_actual->nodo_izq != nullptr)
00348             {
00349                 cola.push(nodo_actual->nodo_izq);
00350             }
00351
00352             if (nodo_actual->nodo_der != nullptr)
00353             {
00354                 cola.push(nodo_actual->nodo_der);
00355             }
00356         }
00357
00358         cout << endl;
00359         nivel++;
00360     }
00361 }
```

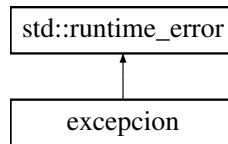
The documentation for this class was generated from the following files:

- [headers/arbol.h](#)
- [src/arbol.cpp](#)

8.2 excepcion Class Reference

```
#include <excepcion.h>
```

Inheritance diagram for excepcion:



Public Member Functions

- [excepcion](#) (const std::string &mensaje)

8.2.1 Detailed Description

Definition at line 15 of file [excepcion.h](#).

8.2.2 Constructor & Destructor Documentation

8.2.2.1 excepcion()

```
excepcion::excepcion (
    const std::string & mensaje) [inline], [explicit]
```

Definition at line 18 of file [excepcion.h](#).

```
00018 : std::runtime_error(mensaje) {}
```

The documentation for this class was generated from the following file:

- headers/[excepcion.h](#)

8.3 Nodo< T > Class Template Reference

```
#include <nodo.h>
```

Public Member Functions

- [Nodo](#) (T dato, [Nodo](#)< T > *nuevo_izq, [Nodo](#)< T > *nuevo_der)
Inicializa el [Nodo](#) con un valor, estableciendo sus punteros a los nodos "izquierdo" y "derecho".
- [Nodo](#) (T dato)
Inicializa el [Nodo](#) con un valor, estableciendo sus punteros a nullptr.
- [Nodo](#) ()
Inicializa el [Nodo](#) vacío (sin valor ni referencias); Es el constructor predeterminado del [Nodo](#).

Public Attributes

- [T valor](#)
- [Nodo< T > * nodo_izq](#)
- [Nodo< T > * nodo_der](#)

8.3.1 Detailed Description

```
template<typename T>
class Nodo< T >
```

Definition at line 14 of file [nodo.h](#).

8.3.2 Constructor & Destructor Documentation**8.3.2.1 Nodo() [1/3]**

```
template<typename T>
Nodo< T >::Nodo (
    T dato,
    Nodo< T > * nuevo_izq,
    Nodo< T > * nuevo_der)
```

Inicializa el [Nodo](#) con un valor, estableciendo sus punteros a los nodos "izquierdo" y "derecho".

Parameters

<i>dato</i>	El valor que se almacenará en el Nodo .
<i>nuevo_izq</i>	La dirección de memoria del nodo conectado a este por el lado izquierdo.
<i>nuevo_der</i>	La dirección de memoria del nodo conectado a este por el lado derecho.

Definition at line 19 of file [nodo.cpp](#).

```
00020 {
00021     valor = dato;
00022     nodo_izq = nuevo_izq;
00023     nodo_der = nuevo_der;
00024 }
```

8.3.2.2 Nodo() [2/3]

```
template<typename T>
Nodo< T >::Nodo (
    T dato)
```

Inicializa el [Nodo](#) con un valor, estableciendo sus punteros a nullptr.

Parameters

<i>dato</i>	El valor que se almacenará en el Nodo .
-------------	---

Definition at line 35 of file [nodo.cpp](#).

```
00036 {
00037     valor = dato;
00038     nodo_izq = nullptr;
00039     nodo_der = nullptr;
00040 }
```

8.3.2.3 **Nodo()** [3/3]

```
template<typename T>
Nodo< T >::Nodo ()
```

Inicializa el **Nodo** vacío (sin valor ni referencias); Es el constructor predeterminado del **Nodo**.

Inicializa el **Nodo** vacío (sin valor ni referencias). Es el constructor predeterminado del **Nodo**.

Definition at line 50 of file [nodo.cpp](#).

```
00051 {
00052     valor = T();
00053     nodo_izq = nullptr;
00054     nodo_der = nullptr;
00055 }
```

8.3.3 Member Data Documentation

8.3.3.1 **nodo_der**

```
template<typename T>
Nodo<T>* Nodo< T >::nodo_der
```

Definition at line 19 of file [nodo.h](#).

8.3.3.2 **nodo_izq**

```
template<typename T>
Nodo<T>* Nodo< T >::nodo_izq
```

Definition at line 18 of file [nodo.h](#).

8.3.3.3 **valor**

```
template<typename T>
T Nodo< T >::valor
```

Definition at line 17 of file [nodo.h](#).

The documentation for this class was generated from the following files:

- [headers/nodo.h](#)
- [src/nodo.cpp](#)

Chapter 9

File Documentation

9.1 headers/arbol.h File Reference

Define la estructura del árbol binario.

```
#include <string>
#include "nodo.h"
#include "../src/arbol.cpp"
```

Classes

- class [Arbol< T >](#)

9.1.1 Detailed Description

Define la estructura del árbol binario.

Su estructura consta de un puntero que referencia a la raíz del árbol, del cual se pueden derivar nodos internos y hojas.

Este archivo define los constructores y métodos de la plantilla [Arbol](#).

Definition in file [arbol.h](#).

9.2 arbol.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #include <string>
00003 #include "nodo.h"
00004
00013
00014 template <typename T>
00015 class Arbol
00016 {
00017     private:
00018         Nodo<T>* raiz;
00019         void eliminarArbol(Nodo<T>* inicio);
00020
00021         int contarNodos(Nodo<T>* inicio);
00022         int contarHojas(Nodo<T>* inicio);
00023         int contarInternos(Nodo<T>* inicio, Nodo<T>* arbol_raiz);
00024         double sumaNodos(Nodo<T>* inicio);
00025         T buscarMax(Nodo<T>* inicio);
00026
00027         void ordenPre(Nodo<T>* inicio);
00028         void ordenIn(Nodo<T>* inicio);
00029         void ordenPost(Nodo<T>* inicio);
00030
00031     public:
00032         Arbol();
00033         ~Arbol();
00034
00035         void insertarDato(T dato);
00036
00037         int contarNodos();
00038         int contarHojas();
00039         int contarInternos();
00040         T buscarMax();
00041         double promedio(); //< Calcula el promedio de los valores almacenados en los nodos del árbol.
00042         void visualizacion();
00043         void orden();
00044 };
00045
00046 #include "../src/arbol.cpp"

```

9.3 headers/excepcion.h File Reference

Define la clase "excepción". La clase excepción permite la gestión de excepciones que pueda presentar la lista.

```

#include <exception>
#include <string>

```

Classes

- class [excepcion](#)

9.3.1 Detailed Description

Define la clase "excepción". La clase excepción permite la gestión de excepciones que pueda presentar la lista.

Se define una función en excepción capaz de lanzar un mensaje de error personalizado a través del parámetro "mensaje". Dentro de [arbol.cpp](#), esta función es implementada con un "throw excepcion".

Definition in file [excepcion.h](#).

9.4 excepcion.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include <exception>
00003 #include <string>
00004
00014
00015 class excepcion : public std::runtime_error
00016 {
00017     public:
00018         explicit excepcion(const std::string& mensaje) : std::runtime_error(mensaje) {}
00019 };
```

9.5 headers/nodo.h File Reference

Define la estructura de un [Nodo](#).

```
#include <string>
#include "../src/nodo.cpp"
```

Classes

- class [Nodo< T >](#)

9.5.1 Detailed Description

Define la estructura de un [Nodo](#).

Su estructura consta de un valor tipo T y dos punteros que referencian a los nodos conectados a este, es decir, los nodos "izquierdo" y "derecho".

Este archivo define los constructores y métodos de la plantilla [Nodo](#).

Definition in file [nodo.h](#).

9.6 nodo.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include <string>
00003
00012
00013 template <typename T>
00014 class Nodo
00015 {
00016     public:
00017         T valor;
00018         Nodo<T>* nodo_izq;
00019         Nodo<T>* nodo_der;
00020
00021         Nodo(T dato, Nodo<T>* nuevo_izq, Nodo<T>* nuevo_der);
00022         Nodo(T dato);
00023         Nodo();
00024 };
00025
00026 #include "../src/nodo.cpp"
```

9.7 README.md File Reference

9.8 src/arbol.cpp File Reference

Implementación de los constructores y métodos de la plantilla [Arbol](#).

```
#include <iostream>
#include <queue>
#include "arbol.h"
#include "nodo.h"
#include "excepcion.h"
```

9.8.1 Detailed Description

Implementación de los constructores y métodos de la plantilla [Arbol](#).

Definition in file [arbol.cpp](#).

9.9 arbol.cpp

[Go to the documentation of this file.](#)

```
00001 #include <iostream>
00002 #include <queue>
00003 #include "arbol.h"
00004 #include "nodo.h"
00005 #include "excepcion.h"
00006
00011
00012 using namespace std;
00013
00014 /*****
00015
00020
00021 template <typename T>
00022 Arbol<T>::Arbol()
00023 {
00024     raiz = nullptr;
00025 }
00026
00027 /*****
00028
00034
00035 template <typename T>
00036 void Arbol<T>::eliminarArbol(Nodo<T>* inicio)
00037 {
00038     if (inicio != nullptr)
00039     {
00040         eliminarArbol(inicio->nodo_izq);
00041         eliminarArbol(inicio->nodo_der);
00042         delete inicio;
00043     }
00044 }
00045
00051
00052 template <typename T>
00053 Arbol<T>::~~Arbol()
00054 {
00055     eliminarArbol(raiz);
00056 }
00057
00058 /*****
00059
00068
00069 template <typename T>
00070 void Arbol<T>::insertarDato(T dato)
```



```

00071 {
00072     Nodo<T>* nuevo_nodo = new Nodo<T>(dato);
00073     if (raiz == nullptr)
00074     {
00075         raiz = nuevo_nodo;
00076     }
00077     else
00078     {
00079         Nodo<T>* anterior = nullptr;
00080         Nodo<T>* inicio = raiz;
00081
00082         while(inicio != nullptr)
00083         {
00084             anterior = inicio;
00085             inicio = (dato < inicio->valor) ? inicio->nodo_izq : inicio->nodo_der;
00086         }
00087
00088         if (dato < anterior->valor)
00089         {
00090             anterior->nodo_izq = nuevo_nodo;
00091         }
00092         else
00093         {
00094             anterior->nodo_der = nuevo_nodo;
00095         }
00096     }
00097
00098     cout << "¡Se ha añadido con éxito el nodo con valor " << dato << "!\n";
00099 }
00100
00101 /*****
00102
00103
00104
00105
00106
00107
00108
00109 template <typename T>
00110 int Arbol<T>::contarNodos(Nodo<T>* inicio)
00111 {
00112     if (inicio == nullptr)
00113     {
00114         return 0;
00115     }
00116
00117     return 1 + contarNodos(inicio->nodo_izq) + contarNodos(inicio->nodo_der);
00118 }
00119
00120
00121
00122
00123
00124 template <typename T>
00125 int Arbol<T>::contarNodos()
00126 {
00127     return contarNodos(raiz);
00128 }
00129
00130 /*****
00131
00132
00133
00134
00135
00136
00137
00138
00139
00140
00141 template <typename T>
00142 int Arbol<T>::contarHojas(Nodo<T>* inicio)
00143 {
00144     if (inicio == nullptr)
00145     {
00146         return 0;
00147     }
00148     else if (inicio->nodo_izq == nullptr && inicio->nodo_der == nullptr)
00149     {
00150         return 1;
00151     }
00152     else
00153     {
00154         return contarHojas(inicio->nodo_izq) + contarHojas(inicio->nodo_der);
00155     };
00156 }
00157
00158
00159
00160
00161
00162 template <typename T>
00163 int Arbol<T>::contarHojas()
00164 {
00165     return contarHojas(raiz);
00166 }
00167
00168 /*****
00169
00170
00171
00172
00173
00174
00175
00176
00177
00178
00179
00180
00181 template <typename T>
00182 int Arbol<T>::contarInternos(Nodo<T>* inicio,  Nodo<T>* arbol_raiz)
00183 {
00184     if (inicio == nullptr)
00185     {
00186         return 0;
00187     }

```

```

00188     }
00189
00190     int nodos = 0;
00191
00192     if ((inicio->nodo_izq != nullptr || inicio->nodo_der != nullptr) && inicio != arbol_raiz)
00193     {
00194         nodos = 1;
00195     }
00196
00197     return nodos + contarInternos(inicio->nodo_izq, arbol_raiz) + contarInternos(inicio->nodo_der,
arbol_raiz);
00198 }
00199
00200
00201
00202
00203
00204 template <typename T>
00205 int Arbol<T>::contarInternos()
00206 {
00207     return contarInternos(raiz, raiz);
00208 }
00209
00210 /*****
00211
00212
00213
00214
00215
00216
00217
00218
00219
00220
00221
00222
00223
00224
00225 template <typename T>
00226 T Arbol<T>::buscarMax(Nodo<T>* inicio)
00227 {
00228     if (inicio == nullptr)
00229     {
00230         throw excepcion(";El árbol se encuentra vacío!");
00231     }
00232
00233     while (inicio->nodo_der != nullptr)
00234     {
00235         inicio = inicio->nodo_der;
00236     }
00237
00238     return inicio->valor;
00239 }
00240
00241
00242
00243
00244
00245 template <typename T>
00246 T Arbol<T>::buscarMax()
00247 {
00248     return buscarMax(raiz);
00249 }
00250
00251 /*****
00252
00253
00254
00255
00256
00257
00258
00259
00260
00261
00262
00263
00264
00265
00266
00267
00268
00269
00270 /*****
00271
00272
00273
00274
00275
00276
00277
00278
00279
00280 template <typename T>
00281 double Arbol<T>::promedio()
00282 {
00283     if (raiz == nullptr)
00284     {
00285         throw excepcion(";El árbol se encuentra vacío!");
00286     }
00287
00288     double suma = sumaNodos(raiz);
00289     double num = contarNodos();
00290
00291     return suma / num;
00292 }
00293
00294 /*****
00295
00296
00297
00298
00299
00300
00301
00302
00303
00304
00305
00306 template <typename T>
00307 void Arbol<T>::visualizacion()
00308 {
00309     if (raiz == nullptr)
00310     {
00311         throw excepcion(";El árbol se encuentra vacío!");
00312     }

```

```

00313
00314     queue<Nodo<T>*> cola;
00315     cola.push(raiz);
00316
00317     int nivel = 0;
00318
00319     while (!cola.empty())
00320     {
00321         int elementosNivel = cola.size();
00322         cout << "Nivel " << nivel << " = ";
00323
00324         for (int i = 0; i < elementosNivel; i++)
00325         {
00326
00327             Nodo<T>* nodo_actual = cola.front();
00328             cola.pop();
00329
00330             string categoria;
00331
00332             if (nodo_actual == raiz)
00333             {
00334                 categoria = "(Raíz) ";
00335             }
00336             else if (nodo_actual->nodo_izq == nullptr && nodo_actual->nodo_der == nullptr)
00337             {
00338                 categoria = "(Hoja) ";
00339             }
00340             else
00341             {
00342                 categoria = "(Interno)";
00343             }
00344
00345             cout << categoria << " " << nodo_actual->valor << " | ";
00346
00347             if (nodo_actual->nodo_izq != nullptr)
00348             {
00349                 cola.push(nodo_actual->nodo_izq);
00350             }
00351
00352             if (nodo_actual->nodo_der != nullptr)
00353             {
00354                 cola.push(nodo_actual->nodo_der);
00355             }
00356         }
00357
00358         cout << endl;
00359         nivel++;
00360     }
00361 }
00362
00363 /*****
00364
00371
00372 template <typename T>
00373 void Arbol<T>::ordenPre(Nodo<T>* inicio)
00374 {
00375     if (inicio != nullptr)
00376     {
00377         cout << inicio->valor << " | ";
00378         ordenPre(inicio->nodo_izq);
00379         ordenPre(inicio->nodo_der);
00380     }
00381 }
00382
00383 /*****
00384
00393
00394 template <typename T>
00395 void Arbol<T>::ordenIn(Nodo<T>* inicio)
00396 {
00397     if (inicio != nullptr)
00398     {
00399         ordenPre(inicio->nodo_izq);
00400         cout << inicio->valor << " | ";
00401         ordenPre(inicio->nodo_der);
00402     }
00403 }
00404
00405 /*****
00406
00415
00416 template <typename T>
00417 void Arbol<T>::ordenPost(Nodo<T>* inicio)
00418 {
00419     if (inicio != nullptr)
00420     {
00421         ordenPre(inicio->nodo_izq);

```

```

00422         ordenPre(inicio->nodo_der);
00423         cout << inicio->valor << " | ";
00424     }
00425 }
00426
00427 /*****
00428
00429
00430
00431
00432
00433
00434
00435
00436
00437
00438
00439 template <typename T>
00440 void Arbol<T>::orden()
00441 {
00442     if (raiz == nullptr)
00443     {
00444         throw excepcion("El árbol se encuentra vacío!");
00445     }
00446
00447     cout << "\n--- PRE-ORDEN ---\n";
00448     ordenPre(raiz);
00449
00450     cout << "\n\n--- IN-ORDEN ---\n";
00451     ordenIn(raiz);
00452
00453     cout << "\n\n--- POST-ORDEN ---\n";
00454     ordenPost(raiz);
00455 }
00456

```

9.10 src/main.cpp File Reference

El programa completo del árbol binario.

```

#include <iostream>
#include "nodo.h"
#include "arbol.h"

```

Functions

- int [menu](#) ()
Menú del programa que maneja el input (aportación) del usuario.
- int [main](#) ()

9.10.1 Detailed Description

El programa completo del árbol binario.

Las opciones son las siguientes:

- Opción: 1 Insertar un nodo del valor que el usuario desee.
- Opción 2: Mostrar el número total de nodos, y el número de nodos internos.
- Opción 3: Mostrar el promedio de los valores de los nodos, así como el valor máximo.
- Opción 4: Mostrar la categoría y valor de los nodos por nivel (de 0 en adelante).
- Opción 5: Mostrar el contenido del árbol en pre-orden, in-orden y post-orden.
- Opción 6: Salir del programa.

Definition in file [main.cpp](#).

9.10.2 Function Documentation

9.10.2.1 main()

```
int main ()
```

< Inicializa un árbol de tipo "int".

Definition at line 21 of file [main.cpp](#).

```
00022 {
00023     Arbol<int> arbol;
00024     int info;
00025
00026     while (true)
00027     {
00028         switch (menu())
00029         {
00030             case 1:
00031             {
00032                 cout << "\n¿Qué valor desea agregar al árbol?: ";
00033                 cin >> info;
00034
00035                 arbol.insertarDato(info);
00036                 break;
00037             }
00038             case 2:
00039             {
00040                 int aspecto1 = arbol.contarNodos();
00041
00042                 if (aspecto1 == 0)
00043                 {
00044                     cout << "\n;El árbol se encuentra vacío!";
00045                 }
00046                 else
00047                 {
00048                     int aspecto2 = arbol.contarInternos();
00049
00050                     cout << "\nNúmero total de NODOS: " << aspecto1;
00051                     cout << "\nNúmero total de NODOS INTERNOS: " << aspecto2;
00052                 }
00053                 break;
00054             }
00055             case 3:
00056             {
00057                 try
00058                 {
00059                     int aspecto1 = arbol.buscarMax();
00060                     double aspecto2 = arbol.promedio();
00061
00062                     cout << "\nValor MÁXIMO en el árbol: " << aspecto1;
00063                     cout << "\nPROMEDIO de los nodos : " << aspecto2;
00064                 }
00065                 catch(const std::exception& e)
00066                 {
00067                     cerr << e.what() << "\n";
00068                 }
00069                 break;
00070             }
00071             case 4:
00072             {
00073                 try
00074                 {
00075                     arbol.visualizacion();
00076                 }
00077                 catch(const std::exception& e)
00078                 {
00079                     cerr << e.what() << "\n";
00080                 }
00081                 break;
00082             }
00083             case 5:
00084             {
00085                 try
00086                 {
00087                     arbol.orden();
00088                 }
00089                 catch(const std::exception& e)
00090                 {
00091                     cerr << e.what() << "\n";
00092                 }
00093                 break;
00094             }
00095         }
00096     }
00097 }
```

```

00094     }
00095     case 6:
00096     {
00097         cout << "\nSaliendo del programa...";
00098         exit(0);
00099         break;
00100     }
00101     }
00102 }
00103 }

```

9.10.2.2 menu()

```
int menu ()
```

Menú del programa que maneja el input (aportación) del usuario.

Devuelve una opción a la función [main\(\)](#).

Definition at line 113 of file [main.cpp](#).

```

00114 {
00115     int opcion;
00116     do
00117     {
00118         cout << "\n\n\n\t--- MENU! ---";
00119         cout << "\n\n1) Insertar un nodo.\n";
00120         cout << "2) Conteo de estructura (Total de nodos y de nodos internos).\n";
00121         cout << "3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).\n";
00122         cout << "4) Visualización selectiva (Información de los nodos hojas y de internos).\n";
00123         cout << "5) Visualización en pre-orden, in-orden y post-orden.\n";
00124         cout << "6) Salir del programa.\n";
00125         cout << "\n--> Seleccione una opción: ";
00126         cin >> opcion;
00127     } while (opcion < 1 || opcion > 6);
00128     return opcion;
00129 }

```

9.11 main.cpp

[Go to the documentation of this file.](#)

```

00001 #include <iostream>
00002 #include "nodo.h"
00003 #include "arbol.h"
00004
00017
00018 int menu();
00019 using namespace std;
00020
00021 int main()
00022 {
00023     Arbol<int> arbol;
00024     int info;
00025
00026     while (true)
00027     {
00028         switch (menu())
00029         {
00030             case 1:
00031             {
00032                 cout << "\n¿Qué valor desea agregar al árbol?: ";
00033                 cin >> info;
00034
00035                 arbol.insertarDato(info);
00036                 break;
00037             }
00038             case 2:
00039             {
00040                 int aspecto1 = arbol.contarNodos();
00041
00042                 if (aspecto1 == 0)
00043                 {
00044                     cout << "\n;El árbol se encuentra vacío!";
00045                 }
00046             }
00047         }
00048     }
00049 }

```

```

00046         else
00047         {
00048             int aspecto2 = arbol.contarInternos();
00049
00050             cout << "\nNúmero total de NODOS: " << aspecto1;
00051             cout << "\nNúmero total de NODOS INTERNOS: " << aspecto2;
00052         }
00053         break;
00054     }
00055     case 3:
00056     {
00057         try
00058         {
00059             int aspecto1 = arbol.buscarMax();
00060             double aspecto2 = arbol.promedio();
00061
00062             cout << "\nValor MÁXIMO en el árbol: " << aspecto1;
00063             cout << "\nPROMEDIO de los nodos : " << aspecto2;
00064         }
00065         catch(const std::exception& e)
00066         {
00067             cerr << e.what() << "\n";
00068         }
00069         break;
00070     }
00071     case 4:
00072     {
00073         try
00074         {
00075             arbol.visualizacion();
00076         }
00077         catch(const std::exception& e)
00078         {
00079             cerr << e.what() << "\n";
00080         }
00081         break;
00082     }
00083     case 5:
00084     {
00085         try
00086         {
00087             arbol.orden();
00088         }
00089         catch(const std::exception& e)
00090         {
00091             cerr << e.what() << "\n";
00092         }
00093         break;
00094     }
00095     case 6:
00096     {
00097         cout << "\nSaliendo del programa...";
00098         exit(0);
00099         break;
00100     }
00101 }
00102 }
00103 }
00104
00105 /*****
00106
00112
00113 int menu()
00114 {
00115     int opcion;
00116     do
00117     {
00118         cout << "\n\n\t--- MENU! ---";
00119         cout << "\n\n1) Insertar un nodo.\n";
00120         cout << "2) Conteo de estructura (Total de nodos y de nodos internos).\n";
00121         cout << "3) Análisis de datos (Valor máximo y promedio aritmético de los nodos).\n";
00122         cout << "4) Visualización selectiva (Información de los nodos hojas y de internos).\n";
00123         cout << "5) Visualización en pre-orden, in-orden y post-orden.\n";
00124         cout << "6) Salir del programa.\n";
00125         cout << "\n--> Seleccione una opción: ";
00126         cin >> opcion;
00127     } while (opcion < 1 || opcion > 6);
00128     return opcion;
00129 }
00130
00131 /*****
00132

```

9.12 src/nodo.cpp File Reference

Implementación de los constructores y métodos de la plantilla [Nodo](#).

```
#include "nodo.h"
```

9.12.1 Detailed Description

Implementación de los constructores y métodos de la plantilla [Nodo](#).

Definition in file [nodo.cpp](#).

9.13 nodo.cpp

[Go to the documentation of this file.](#)

```
00001 #include "nodo.h"
00002
00007
00008 /*****
00009
00017
00018 template <typename T>
00019 Nodo<T>::Nodo(T dato, Nodo<T>* nuevo_izq, Nodo<T>* nuevo_der)
00020 {
00021     valor = dato;
00022     nodo_izq = nuevo_izq;
00023     nodo_der = nuevo_der;
00024 }
00025
00026 /*****
00027
00033
00034 template <typename T>
00035 Nodo<T>::Nodo(T dato)
00036 {
00037     valor = dato;
00038     nodo_izq = nullptr;
00039     nodo_der = nullptr;
00040 }
00041
00042 /*****
00043
00048
00049 template <typename T>
00050 Nodo<T>::Nodo()
00051 {
00052     valor = T();
00053     nodo_izq = nullptr;
00054     nodo_der = nullptr;
00055 }
```


Index

- ~Arbol
 - Arbol< T >, [18](#)
- Arbol
 - Arbol< T >, [18](#)
- Arbol< T >, [17](#)
 - ~Arbol, [18](#)
 - Arbol, [18](#)
 - buscarMax, [18](#)
 - contarHojas, [18](#)
 - contarInternos, [18](#)
 - contarNodos, [19](#)
 - insertarDato, [19](#)
 - orden, [19](#)
 - promedio, [20](#)
 - visualizacion, [20](#)
- buscarMax
 - Arbol< T >, [18](#)
- contarHojas
 - Arbol< T >, [18](#)
- contarInternos
 - Arbol< T >, [18](#)
- contarNodos
 - Arbol< T >, [19](#)
- excepcion, [22](#)
 - excepcion, [22](#)
- headers Directory Reference, [15](#)
- headers/arbol.h, [25](#), [26](#)
- headers/excepcion.h, [26](#), [27](#)
- headers/nodo.h, [27](#)
- insertarDato
 - Arbol< T >, [19](#)
- main
 - main.cpp, [33](#)
- main.cpp
 - main, [33](#)
 - menu, [34](#)
- menu
 - main.cpp, [34](#)
- Nodo
 - Nodo< T >, [23](#)
- Nodo< T >, [22](#)
 - Nodo, [23](#)
 - nodo_der, [24](#)
 - nodo_izq, [24](#)
 - valor, [24](#)
- nodo_der
 - Nodo< T >, [24](#)
- nodo_izq
 - Nodo< T >, [24](#)
- orden
 - Arbol< T >, [19](#)
- promedio
 - Arbol< T >, [20](#)
- Pruebas, [3](#)
- README.md, [28](#)
- src Directory Reference, [15](#)
- src/arbol.cpp, [28](#)
- src/main.cpp, [32](#), [34](#)
- src/nodo.cpp, [36](#)
- valor
 - Nodo< T >, [24](#)
- visualizacion
 - Arbol< T >, [20](#)
- Árbol Binario en C++, [1](#)